

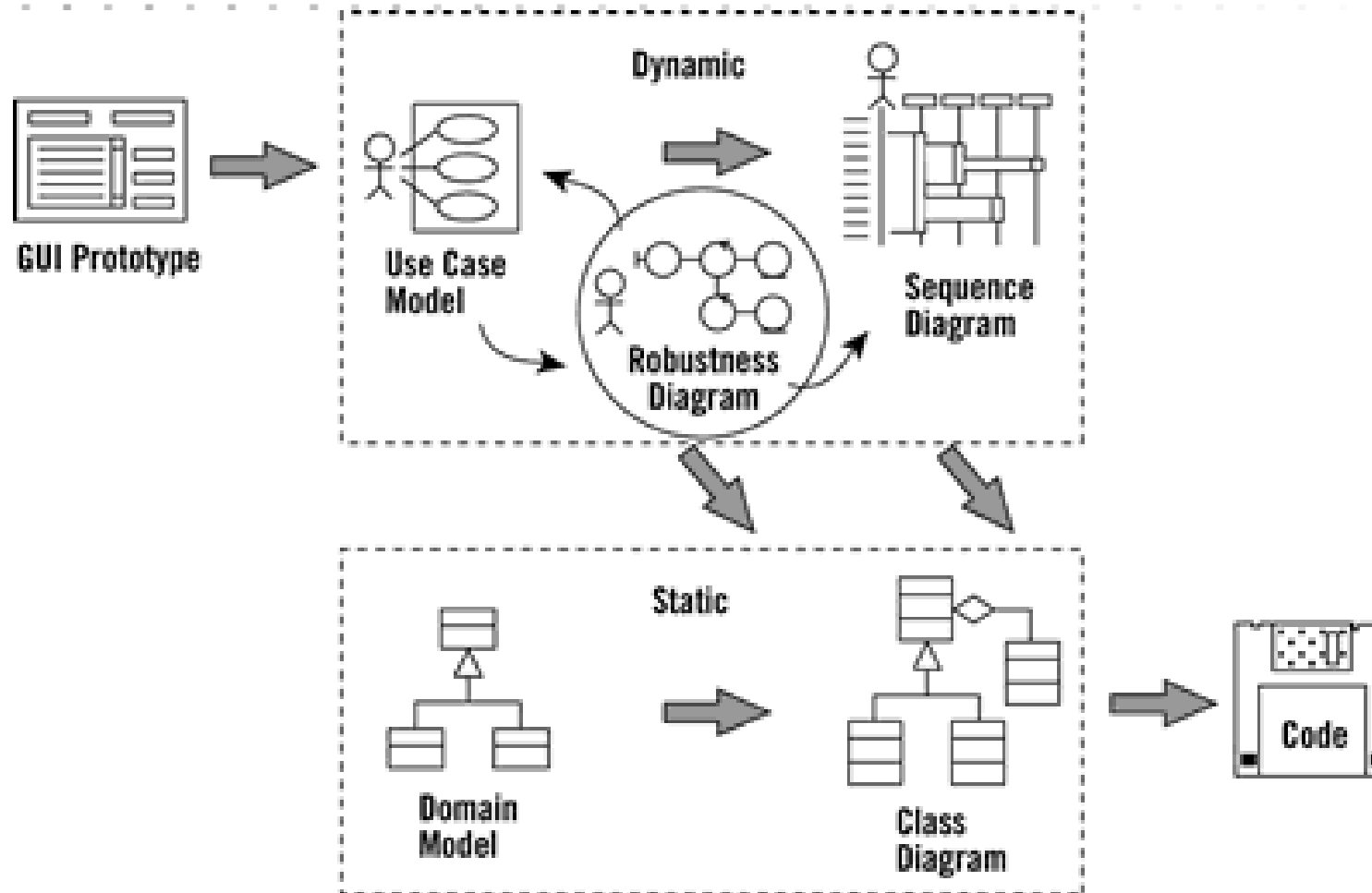
Curso Integrador I

Modelo – Vista - Controlador

Semana 09 - Sesión 1

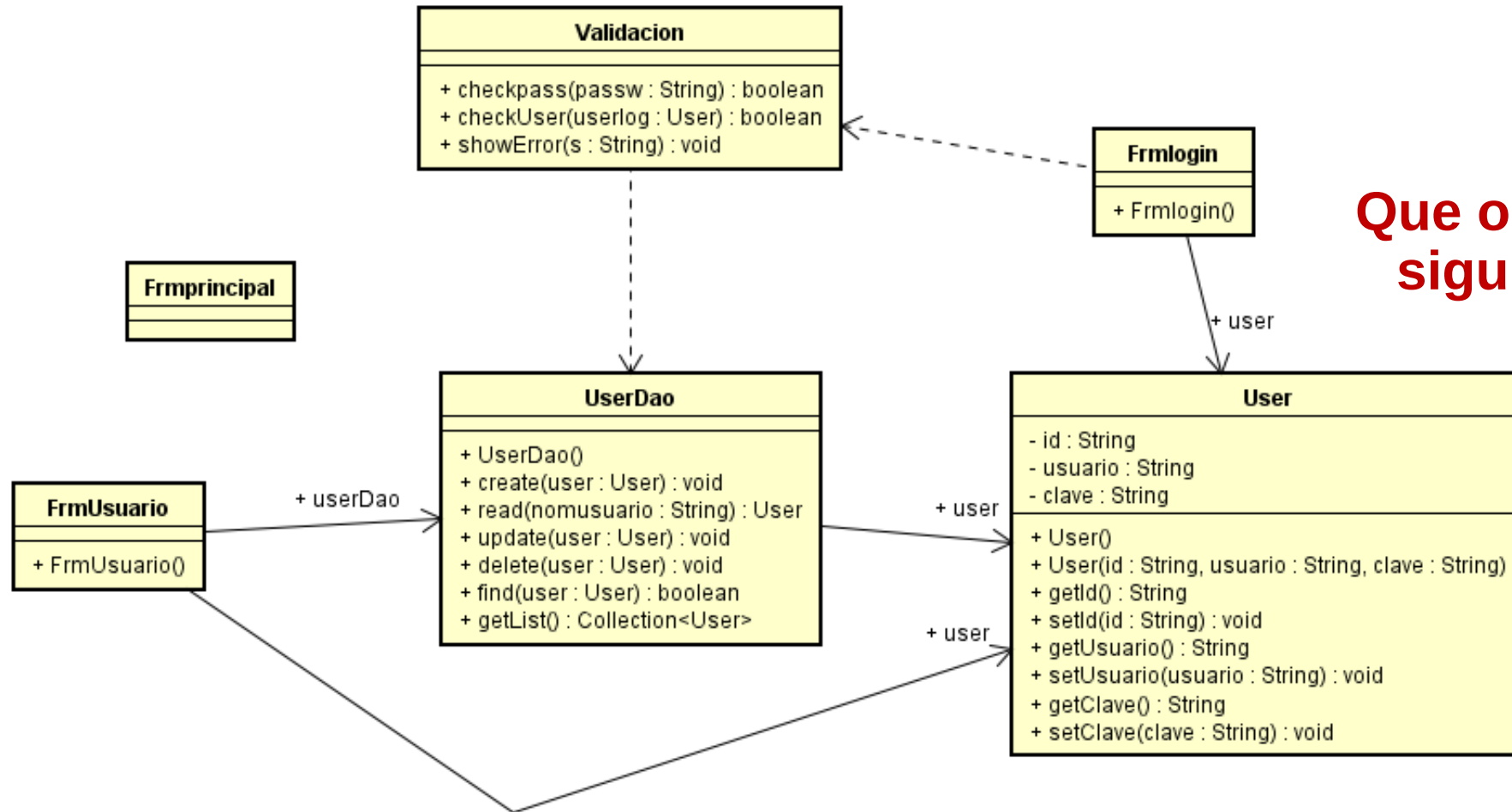
Desaprende lo que te limita

Conocimientos Previos



Desaprende lo que te limita

Dinámica



Que observamos en la siguiente figura?

Desaprende lo que te limita

Logro de la Sesión

Al finalizar la sesión, el estudiante comprende el patrón MVC y lo aplica para implementar soluciones desktop, en el entorno Netbeans.

CONTENIDOS

01

Patrón Modelo -Vista –
Controlador (MVC)

02

Componentes

03

Esquema MVC

04

Ejercicios



Universidad
Tecnológica
del Perú

- Separación clara de dónde tiene que ir cada tipo de lógica, facilitando el mantenimiento y la escalabilidad de nuestra aplicación.
- Facilidad para la realización de pruebas unitarias de los componentes, así como de aplicar desarrollo guiado por pruebas (Test Driven Development o TDD).

Estilo Arquitectónico Modelo – Vista – Controlador



MVC es un estilo arquitectónico que permite dividir un proyecto en capas muy bien definidas: el Modelo, la Vista y el Controlador.

El uso del estándar MVC brinda el beneficio de aislar las reglas de negocio de la lógica de presentación (interfaz de usuario). Esto permite la existencia de varias interfaces de usuario que se pueden modificar sin necesidad de cambiar las reglas de negocio, proporcionando así mucha más flexibilidad y oportunidades para reutilizar clases.

Una de las características del MVC es que se puede aplicar a diferentes sistemas. El estándar MVC se puede utilizar en varios tipos de proyectos, como escritorio, web y móvil.

La comunicación entre interfaces y reglas de negocio se define a través de un controlador, y es la existencia de este lo que hace posible la separación entre capas. Cuando se ejecuta un evento en la interfaz gráfica, como un clic en un botón, la interfaz se comunicará con el controlador, que a su vez se comunica con las reglas de negocio.

Desaprende lo que te limita

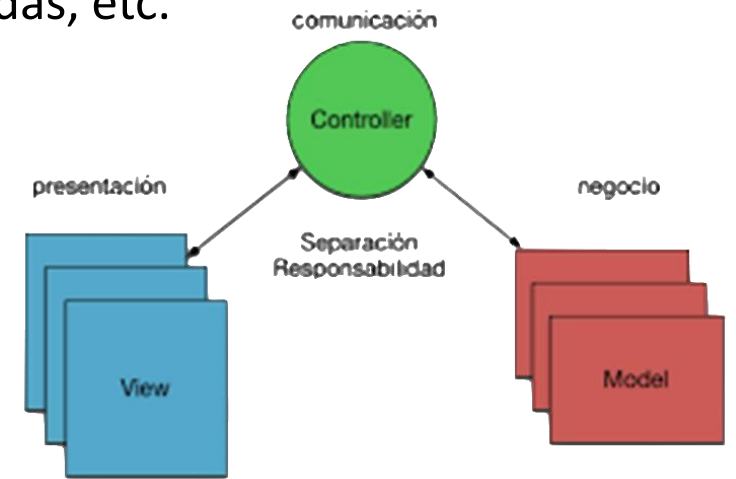
MVC

En MVC cada elemento tiene tres partes:

- Un modelo que contiene los datos y su tratamiento (funcionalidad de la aplicación)
Ej: Juego ajedrez: estado del tablero, reglas del ajedrez, etc.
- Una vista que gestiona como se muestran esos datos
Ej. Juego ajedrez: ventana que dibuja el tablero, oyentes de eventos, etc.
- Un controlador que determina que modificaciones hay que hacer en el modelo cuando se interacciona con la vista (el comportamientos de la aplicación) También puede contener algoritmos
Juego ajedrez: control de eventos, algoritmo para pensar las jugadas, etc.

Acoplamiento: El grado en que una clase conoce a la otra. Si el conocimiento de la clase A sobre la clase B es a través de su interfaz, tenemos un acoplamiento bajo, (Recomendable). Por otro lado, si la clase A depende de miembros de la clase B que no son parte de la interfaz de B, entonces tenemos un alto acoplamiento, (No recomendable).

Cohesión: Cuando tenemos una clase diseñada de tal manera que tiene un propósito único y bien enfocado, decimos que tiene una alta cohesión, (recomendable). Cuando tenemos una clase con propósitos que no le pertenecen solo a ella, tenemos poca cohesión, (no recomendable).



Desaprende lo que te limita

MODELO

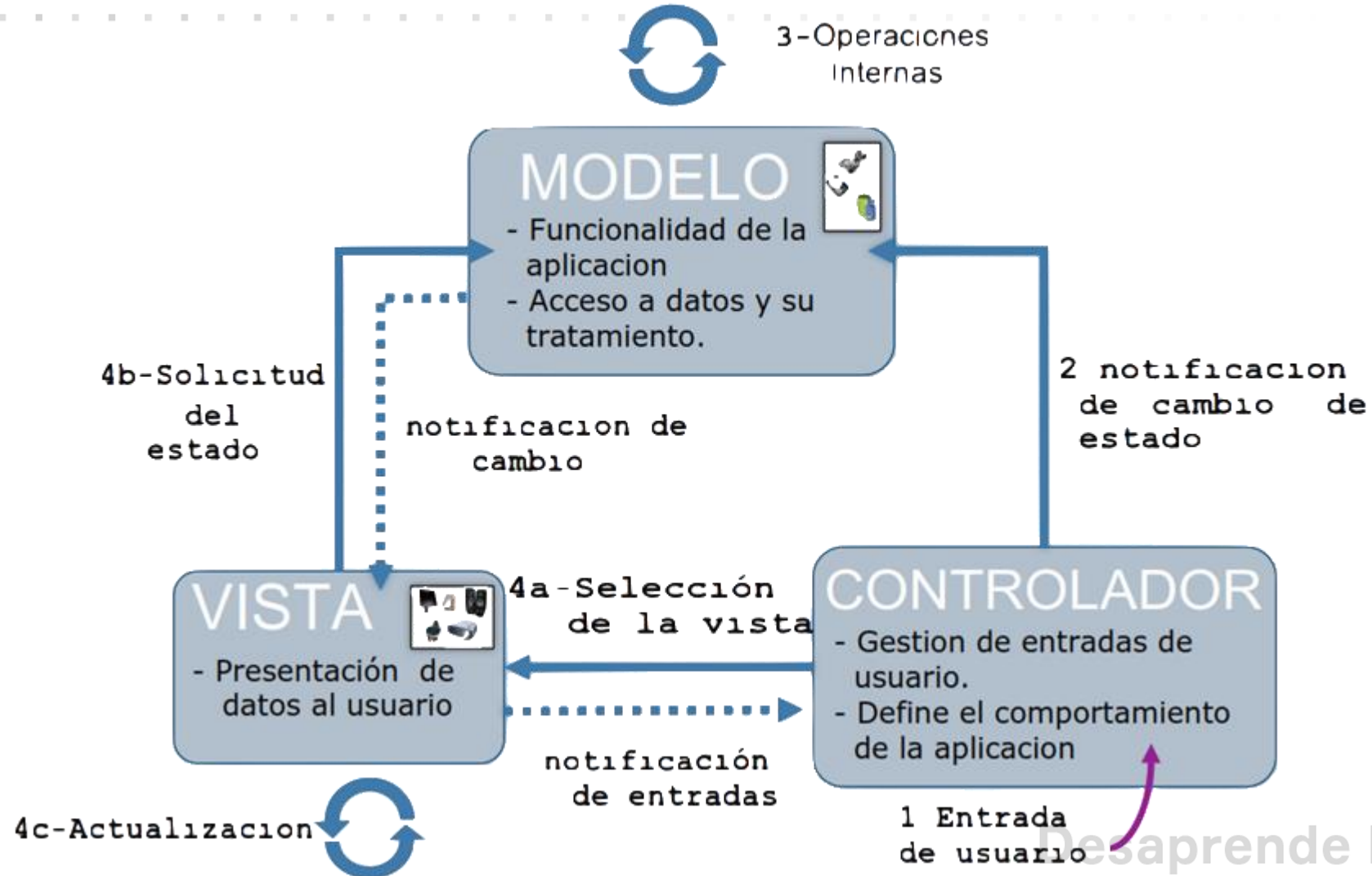
- El modelo es la capa que tiene los datos de la aplicación. Es responsable de las reglas de negocio y las clases de entidad.
- El modelo recibe solicitudes del controlador y genera respuestas a estas solicitudes.
- Siempre que se piense en un modelo , solo conocerá los datos almacenados en el sistema y su lógica. También es en el modelo donde se deben realizar las operaciones CRUD. Esta capa es realmente responsable de la razón por la que se creó la aplicación, es decir, es el núcleo de la aplicación.

- La vista es la capa de visualización y representa la parte del sistema que interactúa con el usuario.
- Es a través de la interfaz que ingresará los datos ingresados por el usuario y también la información que se le mostrará. Estos datos serán insertados o mostrados generalmente por formularios de entrada o salida, tablas, cuadrículas, entre otros formularios.
- La vista no contiene lógica de negocios, por lo que todo el procesamiento lo realiza la capa del modelo y luego el controlador pasa la respuesta a la vista.
- En las aplicaciones de la plataforma Desktop, la visión puede ser representada por clases que se construyen en base a componentes del tipo SWING , AWT , SWT , entre otros. Las aplicaciones de plataforma web, por su parte, estarían representadas por páginas del tipo JSP, JSF , entre otros tipos, que se muestran desde un navegador web. **Desaprende lo que te limita**

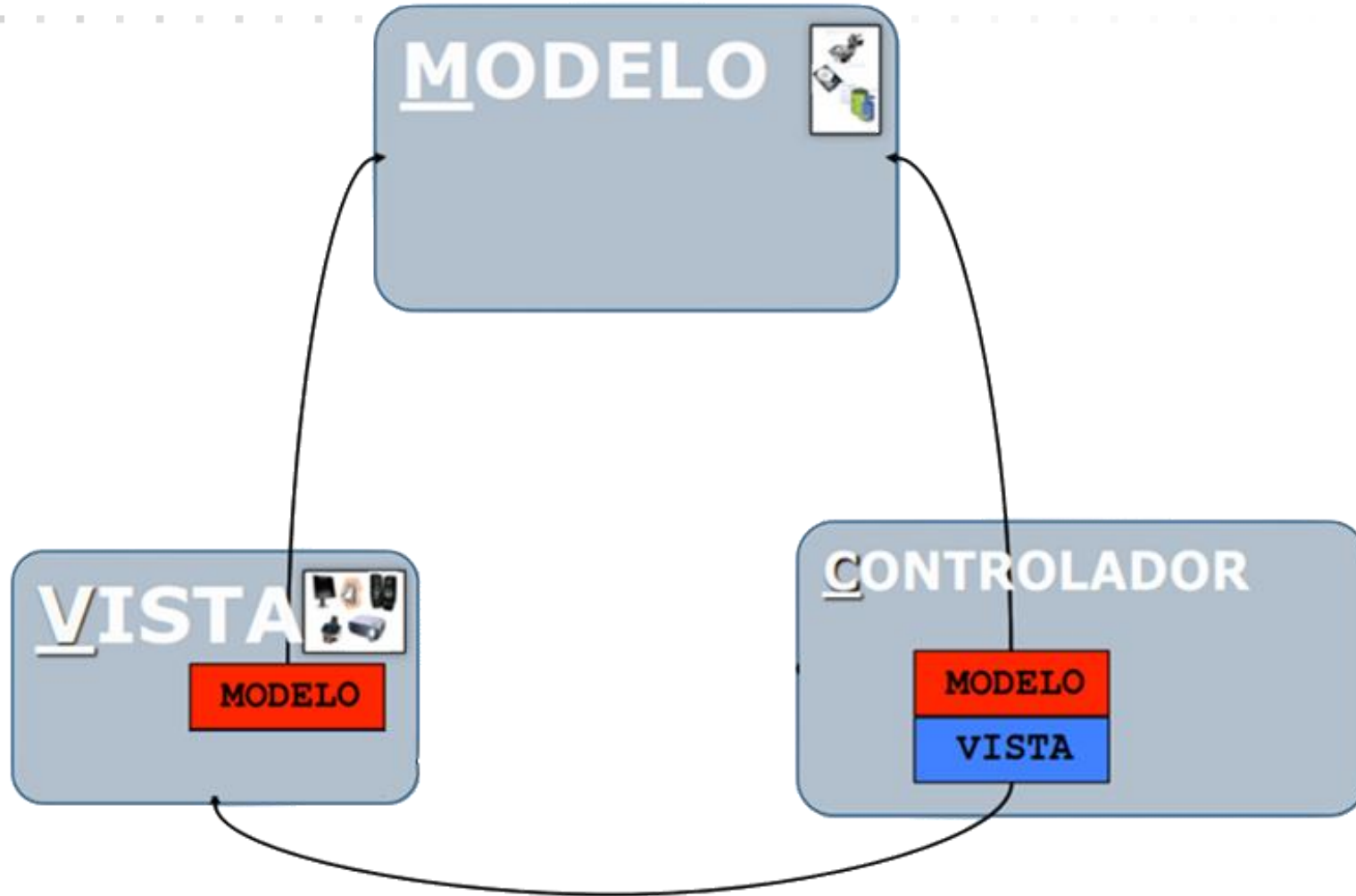
CONTROLADOR

- La función es ser una capa intermedia entre la capa de presentación (Vista) y la capa de negocio (Modelo).
- Es responsable de procesar la solicitud de entrada.
- Recibe la información de los usuarios a través de la vista, procesa los datos del usuario con la ayuda del modelo y devuelve los resultados a la vista.
- De esta manera, cualquier solicitud creada por el usuario debe pasar por el controlador, y el controlador luego se comunica con el modelo. Si el modelo genera una respuesta a estas solicitudes, envía las respuestas al controlador, que a su vez las pasa a la capa de vista .
- El controlador sirve como un intermediario que organiza los eventos de la interfaz de usuario y los dirige a la capa del modelo, por lo que es posible reutilizar la capa del modelo en otros sistemas ya que no hay dependencia entre la visualización y el modelo.

FLUJO DE INFORMACION EN EL MVC



REFERENCIAS ENTRE COMPONENTES



Desaprende lo que te limita

```
//ATRIBUTOS DEL MODELO (Lógica de multiplicar)
```

```
private int op1; //Operando1  
private int op2; //Operando2  
private int res; //Resultado
```

```
//MÉTODOS SET Y GET
```

```
public int getOp1() {  
    return op1;  
}
```

```
public void setOp1(int op1) {  
    this.op1 = op1;  
}
```

```
public int getOp2() {  
    return op2;  
}
```

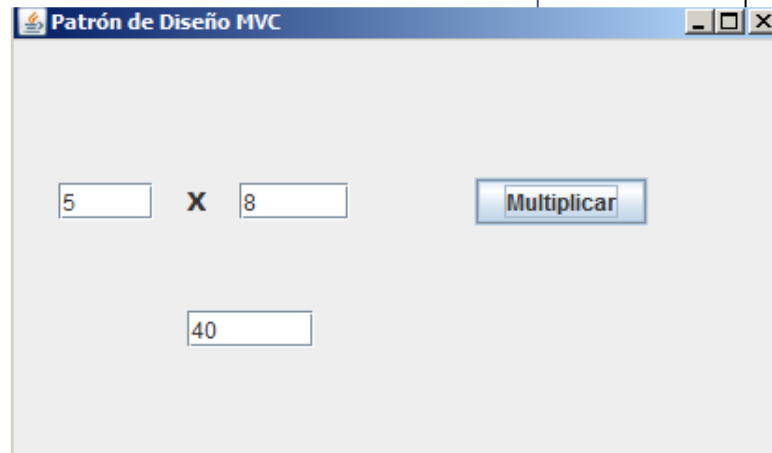
```
public void setOp2(int op2) {  
    this.op2 = op2;  
}
```

```
public int getRes() {  
    return res;  
}
```

```
public void setRes(int res) {  
    this.res = res;  
}
```

```
//MÉTODO FUNCIONAL, EJECUTA LA OPERACIÓN multiplicar
```

```
public int multiplicar(){  
    this.res = this.op1 * this.op2;  
    return this.res;  
}
```



```
//ATRIBUTOS DE CLASE Controlador
```

```
//Controlador ES UNA CLASE COMPUESTA
```

```
private FrmVista view;  
private Modelo model;
```

```
//MÉTODO FUNCIONAL DE Controlador
```

```
public void iniciar(){
```

```
    /**Controlador PUEDE ENVIAR COMANDOS A Vista
```

```
    //ESTABLECER TITULO DE FORMULARIO(Vista)
```

```
    view.setTitle("Patrón de Diseño MVC");
```

```
    //ESTABLECER UBICACIÓN DE FORMULARIO(Vista)
```

```
    view.setLocationRelativeTo(null);
```

```
    //ESTABLECER VISIBILIDAD DE FORMULARIO(Vista)
```

```
    view.setVisible(true);
```

```
}
```

```
@Override
```

```
//IMPLEMENTA MÉTODO DE INTERFACE ActionListener(Oyente)
```

```
public void actionPerformed(ActionEvent ae) {
```

```
    /**Controlador GESTIONA ACTIVIDAD DEL Modelo Y DE LA Vista
```

```
    //ASIGNA OPERANDOS(Modelo) DESDE FORMULARIO(Vista)
```

```
    model.setOp1(Integer.parseInt( view.txtOp1.getText()));
```

```
    model.setOp2(Integer.parseInt( view.txtOp2.getText()));
```

```
    //INVOCA OPERACION MULTIPLICAR(Modelo)
```

```
    model.multiplicar();
```

```
    //MUESTRA RESULTADO(Modelo) EN FORMULARIO(Vista)
```

```
    view.txtRes.setText(String.valueOf(model.getRes()));
```

```
}
```

```
}
```

VENTAJAS DE LA ARQUITECTURA MVC



- Dado que MVC maneja las múltiples vistas usando el mismo modelo empresarial, es más fácil de mantener, probar y actualizar el sistema.
- Será más fácil agregar nuevos clientes con solo agregar sus vistas y controladores.
- Dado que el modelo está completamente desacoplado de la vista, permite una gran flexibilidad para diseñar e implementar el modelo considerando la reutilización y modularidad. Este modelo también puede extenderse para sistemas distribuidos.
- Los cambios en la vista no afectan las reglas de negocio ya implementadas en la capa del modelo;
- Permite el desarrollo, la prueba y el mantenimiento de forma aislada entre capas.
- Es posible tener procesos de desarrollo en paralelo para el modelo, la vista y controlador.
- Esto hace que la aplicación sea extensible y escalable.

DESVENTAJAS DE LA ARQUITECTURA MVC



Universidad
Tecnológica
del Perú

- En sistemas de baja complejidad, MVC puede crear una complejidad innecesaria.
- Requiere mucha disciplina por parte de los desarrolladores con respecto a la separación de las capas;
- Requiere más tiempo modelar el sistema.
- Siempre que se desarrolle un sistema, ciertamente tendrá la necesidad de mantenimiento, como mejoras o corrección de errores, y también puede necesitar varias actualizaciones. Aunque algunos programadores encuentran que el estándar MVC es demasiado costoso de aplicar, se vuelve mucho más fácil cuando hay necesidad de realizar mantenimiento y actualizaciones, ya que tiene una estructura bien definida y capas separadas para reducir la dependencia entre ellos.

Desaprende lo que te limita

Caso de Ejemplo

- Realizar una aplicación en modo consola, que nos permita hacer la conversión de grados celsius a fahrenheit y viceversa

$\text{Celcius} = (\text{fahrenheit} - 32) * (5/9);$

$\text{Fahrenheit} = \text{celcius} * (9/5) + 32;$

Trabajo:

- Representar la lógica del negocio en otra clase del tal manera que la entidad sea un pojo.
- Agregar la siguiente funcionalidad: Conversion de Celsius a Kelvin
- Agregar una clase llamada Unidad que encapsule los tipos de temperatura (Celsius, Fahrenheit, Kelvin). Realizar lo cambios convenientes

Que hemos aprendido hoy



- MVC es un patrón de diseño de software que separa el código por sus diferentes responsabilidades.
- La Vista es el componente que interactúa con el usuario.
- El Controlador suele estar asociado a un componente “Vista”.
- El Modelo representa el aspecto lógico dentro de este patrón de diseño MVC.

Desaprende lo que te limita



Excelente tu
participación

No hay nada como
un reto para sacar lo
mejor de nosotros.



Ésta sesión
quedará
grabada para tus
consultas.



PARATI

1. Sigue practicando,
vamos tu puedes!! .
2. No olvides que
tienes un FORO
para tus consultas.

Desaprende lo que te limita



Universidad
Tecnológica
del Perú



**Universidad
Tecnológica
del Perú**

Desaprende lo que te limita